Architecture: Personalized Learning Path Recommender

Oct 22, 2026 · @rishu

Overview

Build one stateless FastAPI backend as a modular monolith, with PostgreSQL as the system of record and Redis for caching. This meets the SRS targets (1,000 concurrent learners, 500 ms p95 recommendations) without the cost of microservices, and the module boundaries let any service be split out later.

Design principles:

- **Mastery is written on the request path.** FR-302 requires it before the next question or recommendation, and NFR-03 gives 1 second. Everything else (engagement rollups, dashboard aggregates, experiment metrics) runs asynchronously.
- **One strategy interface.** Rule-based, embeddings plus rules, and collaborative filtering all implement the same call, so A/B arms and the rule-based fallback (NFR-08) need no client change.
- **Every recommendation carries its reason.** The reason is produced by the ranker, not added afterwards (FR-505, NFR-11).
- **Configuration lives in the database.** Weights, thresholds, learning rate and strategy switches are admin-editable without a deploy (NFR-13, FR-807).
- **Access is checked twice.** The gateway validates the token and role; each service re-checks ownership (a learner sees only their own data, an instructor only their own courses).

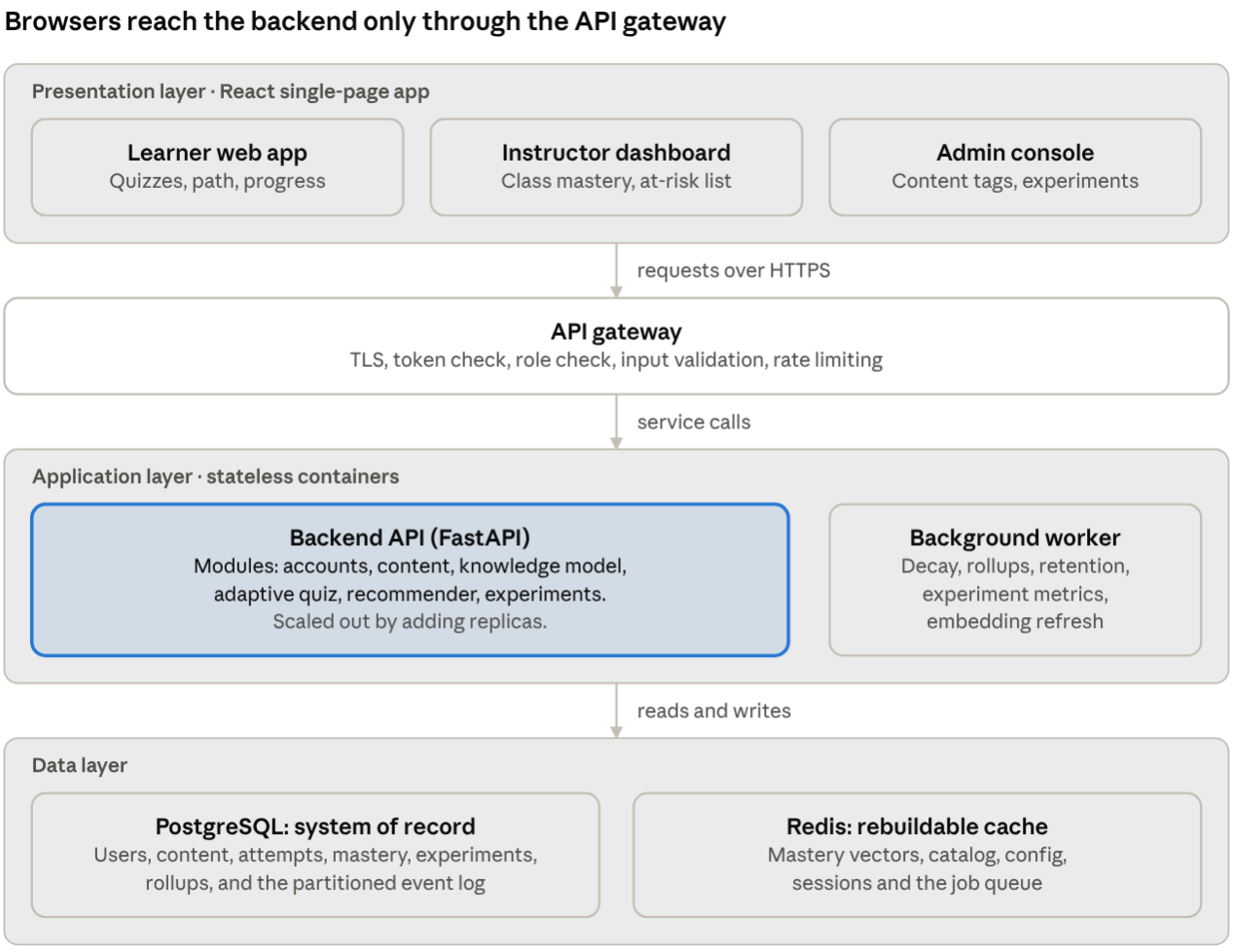
Assumptions taken for the PRD's open questions. Please confirm or correct them.

Open question	Assumption in this document
Standalone app or LMS plug-in	Standalone web app; an LTI launch adapter (EI-5) is a later add-on at the gateway
Tech stack	Python 3.12 + FastAPI, React + TypeScript, PostgreSQL, Redis, Docker
Real or synthetic data	Synthetic learner data generated by a seed script, so the A/B and dashboard features can be demonstrated

Open question	Assumption in this document
Demo subject area	Not fixed; the schema is subject-agnostic and only the content tags change

System architecture

The system has four layers: browser clients, one API gateway, stateless application containers, and a data layer, with clients allowed to reach services only through the gateway.



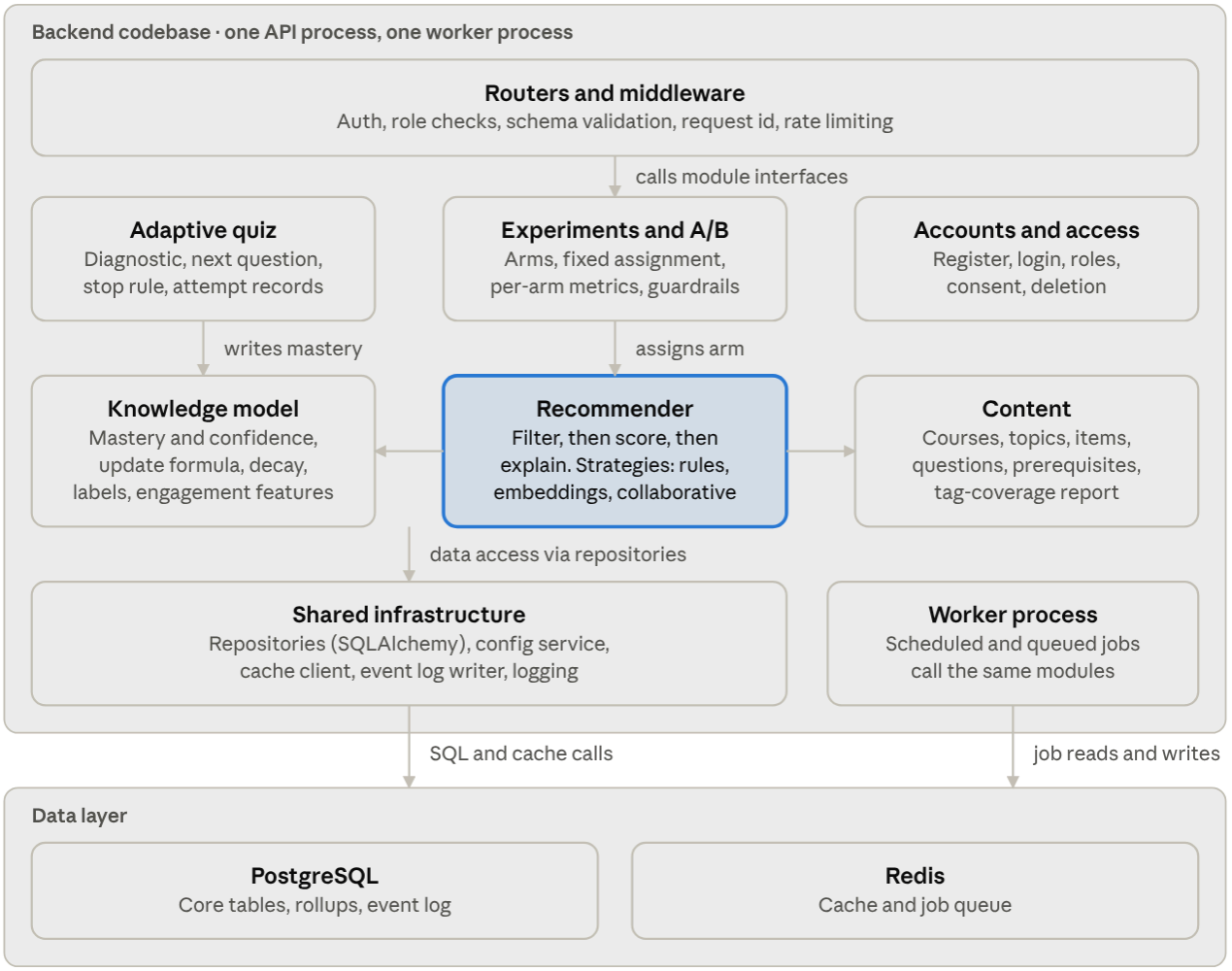
system architecture · 4 layers

The gateway is the only public entry point and services never expose themselves (SRS section 2.1). The backend API and the worker share the same data layer: the API serves requests, and the worker handles anything that need not finish before a response. If the system is later embedded in an LMS, the LTI launch (EI-5) is added at the gateway without touching the services.

Backend architecture

The backend is one FastAPI codebase run as two processes (API and worker), organized into six modules that call each other only through service interfaces.

Six modules share one backend; only the recommender swaps strategies



backend architecture · 6 modules, 1 worker

Arrows between modules show who calls whom. The quiz module sends each graded answer to the knowledge model, which owns the mastery write. The recommender reads mastery from the knowledge model and the catalog from the content module, and takes its strategy from the experiments module. Accounts and access works through the middleware, so no other module handles passwords or tokens.

Each module has the same four files, which keeps the codebase easy to navigate and to test:

- **router.py:** thin HTTP handlers; validate input, call one service function, shape the response.

- **service.py:** the business rules and the module's public interface; the only file other modules may import.
- **repository.py:** the only code that touches the module's tables.
- **schemas.py:** request and response models shared with the OpenAPI description.

```
app/
  main.py              FastAPI app, router registration
  core/                auth, roles, config client, logging, errors
  modules/
    accounts/  content/  knowledge/  quiz/  recommender/  experiments/
      router.py  service.py  repository.py  schemas.py
  recommender/
    strategies/        rules.py  embeddings.py  collaborative.py
  worker/              job definitions and schedules
  db/                  SQLAlchemy models, Alembic migrations
```

Service responsibilities

Six backend modules, a shared config service and a background worker cover every functional group in the SRS. Each module owns its tables and exposes a Python interface to the others; no module writes another module's tables.

Module	Owns	SRS requirements
Accounts and access	Users, roles, consent, tokens, deletion and anonymization	FR-101 to FR-104, NFR-05 to NFR-07
Content	Courses, topics, content items, questions, tags, prerequisites, tag-coverage report	FR-201 to FR-203
Knowledge model	Mastery and confidence per learner-topic, update formula, decay, weak/developing/strong labels, engagement features	FR-301 to FR-307
Adaptive quiz	Diagnostic and adaptive quiz sessions, next-question selection, stop rule, attempts	FR-601 to FR-605
Recommender	Strategy interface, candidate filters, ranking, reasons, dismissals, cold-start fallback	FR-501 to FR-510

Module	Owns	SRS requirements
Experiments and analytics	Experiments, arms, persistent assignment, recommendation log, per-arm metrics, significance, guardrails, dashboard queries, exports	FR-701 to FR-705, FR-801 to FR-807
Config	Weights, thresholds, learning rates, strategy on/off flags, retention period	NFR-13, FR-807
Background worker	Mastery decay, engagement rollups, experiment metrics, retention aggregation, embedding refresh	FR-305, FR-307, FR-805, FR-806, section 5.3

Key runtime flows

Three flows carry the product: answering a question, asking for recommendations, and reading experiment results.

Submit a quiz answer (UC-03)

1. The client sends the answer to the quiz endpoint. The gateway checks the token, role and input schema.
2. The adaptive quiz module grades the answer against the stored correct answer.
3. In one database transaction it inserts the attempt (FR-604), asks the knowledge model to apply the section 7.1 update to the mastery row, and appends an answer event to the event log.
4. After commit, the cached mastery vector for that learner is overwritten, so the next request sees it within 1 second (NFR-03).
5. The module picks the next question: target difficulty is mastery plus stretch, stepped up after 3 correct in a row and down after 2 incorrect, never repeating a question in the session (FR-602, FR-603).
6. If confidence passes the threshold or the maximum length is reached, the quiz ends (FR-605); otherwise the response carries the next question.

Get recommendations (UC-02)

1. The client requests the next items for a course. The gateway authenticates and passes the learner id.

2. The experiments module returns the learner's arm. If none exists and an experiment is running, it draws one at random by the allocation split and stores it permanently (FR-802). With no experiment, the default strategy is used.
3. The recommender loads the mastery vector (Redis first, database on a miss), the cached catalog, and the learner's completed and dismissed items.
4. Hard filters run first: drop completed items unless a review is due, items with weak prerequisites, and dismissed items still in their quiet period (FR-502, FR-503, FR-509).
5. The arm's strategy scores the remaining candidates and returns the top N (default 5), each with a one-line reason. A learner with no history gets diagnostic-based or entry-level items (FR-510).
6. Each recommendation is logged with experiment, arm, rank and reason before the response is sent (FR-803). If the strategy errors or times out, the rule-based strategy answers instead (NFR-08).

Read experiment results

1. A worker job recomputes per-arm acceptance rate, 8-week completion rate and post-test gain on a schedule (FR-804).
2. It computes significance and a confidence interval for each metric, and checks the sample against the 200-learners-per-arm minimum (FR-805, PRD section 7).
3. It compares the guardrail metrics (dropout, quiz abandonment) with their limits and sets the experiment to paused if one is breached (FR-806).
4. The admin console reads the stored results; it never triggers the heavy queries itself.

Recommender internals

The recommender is a fixed pipeline (filter, score, explain) in which only the scoring step is swappable. This keeps A/B tests fair, because all arms share the same filters and the same output shape.

Every strategy implements one interface (FR-506, NFR-12). Adding a strategy means adding one class and registering it; clients do not change.

```
class RecommendationStrategy(Protocol):
    name: str

    def score(self, ctx: LearnerContext, candidates: list[ContentItem]) ->
list[ScoredItem]:
        """Return a score in [0, 1] and the dominant factor for each
candidate."""
```

The pipeline then sorts by score, keeps the top N, and turns each item's dominant factor into the one-line reason. The rule-based strategy uses the SRS section 7.2 formula:

```
S(i) = w_1\,\text{gap}(i) + w_2\,\text{fit}(i) + w_3\,\text{ready}(i) +
w_4\,\text{eng}(i)
```

with default weights 0.4, 0.3, 0.2 and 0.1, read from the config module on each request.

Strategy	Release	How it differs	Extra infrastructure
A: rule-based	MVP	Uses the formula above as written	None
B: embeddings plus rules	v1.1	Similarity from title, description and tags replaces or blends into fit(i) (FR-507)	Embedding vectors stored per content item, refreshed by the worker when tags or text change
C: collaborative filtering	Later	Uses paths that similar learners completed successfully (FR-508)	A batch job that builds learner similarity; needs enough data to be useful

Reasons. The scorer records which weighted term contributed most for each item, and a template turns it into text: a gap term gives "Strengthens a weak topic", a ready term gives "Prerequisites now met", a fit term gives "Matches your current level", an engagement term gives "Format you usually complete". Strategies B and C add their own template (for example "Similar to content you finished"). Templates are editable by admins.

Fallback chain. If the arm's strategy raises an error or exceeds its time budget, the pipeline retries with the rule-based strategy (NFR-08). If the learner has no mastery data at all, it uses the diagnostic result, then entry-level items by difficulty, and popularity once enough completions exist (FR-510, PRD risk table). The response records which step answered, so experiments can exclude fallback traffic from their metrics.

Data architecture

PostgreSQL is the only durable store; Redis holds nothing that cannot be rebuilt from it. The entities and relationships are those of SRS section 5; this section says where each lives and how it is accessed.

Store	Holds	Access notes
PostgreSQL: core tables	Users, courses, topics, content items, questions, attempts, mastery scores, recommendations, experiments, arms, assignments, config	Mastery has a composite key of learner and topic, so one row is read or locked directly. Assignments are keyed by learner and experiment, so a learner's arm can never be duplicated. Attempts are indexed by learner and time; recommendations by learner and time, and by arm for experiment metrics.
PostgreSQL: event log	Opened, completed, skipped, time on task, quiz answered	Append-only table partitioned by month; rows are never updated. Retention drops whole old partitions after the worker aggregates them (SRS section 5.3).
PostgreSQL: rollup tables	Class topic mastery, per-learner completion rate, skip rate and session time, per-arm metrics	Refreshed by the worker so the instructor dashboard reads precomputed rows instead of scanning attempts (NFR-02).
Redis	Learner mastery vector, catalog per course, config snapshot, session state	Mastery is overwritten after each commit; every key has an expiry and a database fallback on a miss.

Concurrent answers. The mastery update reads and writes the learner-topic row under a row lock inside the attempt transaction, so two quick answers cannot overwrite each other.

Deletion and retention. A deletion request removes the account and personal fields, and replaces the learner id on attempts, events and recommendations with an anonymous id, so past experiment results stay valid without identifying anyone. The retention period for events is a setting; its value is still an open question for the institution.

API surface

The backend exposes a versioned REST API with JSON bodies under `/v1`; every endpoint except login and registration needs a token (EI-3). FastAPI generates the OpenAPI description, which also serves as the contract for the React client and the tests.

Role	Endpoint	Purpose	Requirement
Public	`POST /v1/auth/register`, `POST /v1/auth/login`	Create an account, get a token	FR-101
Learner	`POST /v1/me/consent`	Record consent before tracking	FR-104
Learner	`POST /v1/courses/{id}/diagnostic`	Start the diagnostic quiz	FR-601
Learner	`POST /v1/quiz-sessions/{id}/answers`	Submit an answer, get the next question	FR-602, FR-604
Learner	`GET /v1/courses/{id}/recommendations`	Next N items with reasons	FR-501, FR-505
Learner	`POST /v1/recommendations/{id}/dismiss`	Dismiss an item	FR-509
Learner	`POST /v1/content/{id}/events`	Log opened, completed, skipped, time on task	FR-307
Learner	`GET /v1/me/mastery`, `GET /v1/me/progress`	Mastery per topic, completion, time spent	FR-701
Learner	`GET /v1/me/report?format=csv|pdf`	Export visible report	FR-705
Instructor	`GET /v1/instructor/courses/{id}/heatmap`, `.../at-risk`	Class mastery and at-risk list	FR-702, FR-704
Instructor	`GET /v1/instructor/learners/{id}/history`	One learner's attempts and activity	FR-703
Admin	`/v1/admin/courses`, `/topics`, `/content`, `/questions`	Create, edit, retire catalog items	FR-201, FR-202

Role	Endpoint	Purpose	Requirement
Admin	`GET /v1/admin/content/tag-coverage`	Items missing required tags	FR-203
Admin	`POST /v1/admin/experiments`, `GET .../{id}/results`, `POST .../{id}/pause`	Define, read and pause experiments	FR-801, FR-804 to FR-806
Admin	`PUT /v1/admin/strategies/{name}`, `PUT /v1/admin/config`	Switch a strategy, change weights and thresholds	FR-807, NFR-13

Cross-cutting concerns

Five concerns apply to every module, so they sit in shared middleware and libraries rather than in each service.

- **Security.** Passwords use a salted adaptive hash (NFR-05). Tokens are short-lived and expire after inactivity. A role dependency runs on every route, queries are parameterized, and request bodies are validated against schemas (NFR-06). All traffic uses TLS 1.2 or later (EI-7), and login is rate limited.
- **Privacy.** Tracking endpoints refuse requests until the learner has recorded consent (FR-104). Only the fields in the entity model are collected (NFR-07), and deletion works as described under data architecture.
- **Reliability.** A strategy failure falls back to rules (NFR-08). Quiz answers carry a client-generated attempt id, so a retried request cannot count twice. Backups run daily with a tested restore (NFR-09).
- **Configuration.** Weights, thresholds, learning rate, decay and strategy flags are stored in the database, cached in Redis, and changed through the admin API. Each change records who made it and when (NFR-13).
- **Observability.** Structured logs with a request id, latency per endpoint (to check the 500 ms p95 target), the fallback rate per strategy, and guardrail metrics per experiment. Core logic is unit tested, with at least 70% coverage (NFR-12).

Deployment and scaling

The system ships as five container images that run on any Linux host (NFR-14). Only the API and worker scale out; one PostgreSQL primary is enough for the assumed 10,000 learners (NFR-04).

Container	Role	Scaling
Reverse proxy	Terminates TLS, serves the built React app, routes `/v1` to the API	One or two instances
API	FastAPI app with all modules; stateless	Horizontal; add replicas behind the proxy
Worker	Scheduled and queued jobs: decay, rollups, experiment metrics, retention, embeddings	Horizontal, with each job type claimed by one worker at a time
PostgreSQL	Core tables, event log, rollups	Primary with daily backups; add a read replica for dashboards if load needs it
Redis	Cache and job queue	Single instance; safe to lose because everything rebuilds from PostgreSQL

A local Docker Compose file runs all five for development and the demo, with the seed script loading synthetic learners. Staging and production use the same images, differing only in environment variables and secrets. A connection pooler in front of PostgreSQL keeps many API replicas from exhausting connections.

If load grows beyond a single modular backend, the recommender and experiments modules are the first candidates to split into separate services, because they have the clearest interface and the heaviest compute.

Requirement coverage and build order

Each non-functional requirement has a named mechanism in the design, and each is checked by the verification method in the SRS.

Requirement	Mechanism
NFR-01: recommendation within 500 ms at p95	Cached mastery vector and catalog, one scoring pass over filtered candidates, no analytics queries on the request path
NFR-02: instructor dashboard within 3 s for 500 learners	Dashboard reads precomputed rollup tables
NFR-03: mastery visible within 1 s	Updated inside the attempt transaction, cache overwritten on commit

Requirement	Mechanism
NFR-04: horizontal scaling, 10,000 learners	Stateless API replicas; worker replicas; one primary database
NFR-05, NFR-06: security	Shared auth and role middleware, schema validation, parameterized queries
NFR-07: privacy	Consent gate, minimal event schema, anonymizing deletion
NFR-08, NFR-09: reliability	Rule-based fallback, idempotent answers, daily backups
NFR-11: explainability	Reason produced by the ranker for every item
NFR-12, NFR-13: maintainability and configurability	One strategy interface; settings stored in the database
NFR-14: portability	Five container images, no host-specific dependencies

Build order follows the PRD release plan, so that the full loop works first and is measured at Gate 1.

1. **Accounts, consent, content and tagging.** Needed before any tracking (FR-101 to FR-104, FR-201 to FR-203).
2. **Knowledge model, diagnostic and adaptive quiz.** Gives mastery per topic (FR-301 to FR-303, FR-601 to FR-604).
3. **Rule-based recommender with reasons and fallback.** Completes the loop (FR-501 to FR-505, FR-510).
4. **Learner and instructor dashboards with rollups.** (FR-701, FR-702).
5. **Two-arm experiment and strategy switch.** The 50/50 split, recommendation logging and per-arm metrics (FR-801 to FR-804, FR-807).
6. **Gate 1 check.** Load test for p95 under 500 ms and measured acceptance rate against the 60% target.
7. **After Gate 1.** Embeddings strategy, decay job, drill-down, guardrails and significance reporting, then collaborative filtering once data allows.